UNICESUMAR
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
TURMA ADSIS2S-N-A

Karen Andrade Cassero | RA 26005674-2
Lais Santana de Carvalho | RA 26008051-2
Odair Manganaro Junior | RA 26004679-2

AEP (PSE – SAÚDE BUCAL)

Maringá
2026

SUMÁRIO

1. LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO – LINGUAGEM C..... 3
1.1 FUNCIONALIDADES IMPLEMENTADAS.....3
1.2 CÓDIGO DO PROJETO..... 3
2. ENGENHARIA DE SOFTWARE.....15
2.1 REQUISITOS FUNCIONAIS ATUALIZADOS..... 15
2.2 DIAGRAMA DE CASOS DE USO.....16
2.3 DESCRIÇÃO DOS CASOS DE USO PRINCIPAIS.....17
3. PRODUÇÃO TEXTUAL – METEP..... 20
3.1 MANUAL DO USUÁRIO.....20
3.1.1 Apresentação do sistema.....20
3.1.2 Instruções para compilação e execução.....20
3.1.3 Funcionalidades do sistema.....21
3.1.4 Exemplos de utilização e capturas de tela.....21
3.2 CONSIDERAÇÕES FINAIS..... 26
4. GITHUB..... 27
5. VÍDEO..... 27

1. LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO – LINGUAGEM C

1.1 FUNCIONALIDADES IMPLEMENTADAS

O sistema foi implementado em C, com menu repetitivo, estrutura de decisão, vetor de registros, funções com responsabilidades específicas e validação das entradas. Os dados são mantidos em memória durante a execução. O código-fonte está disponível completo no GitHub.

- Realizar login: autenticar o usuário por meio de e-mail, senha e tipo de usuário (diretor da escola, profissional da saúde, professor/equipe pedagógica ou coordenador/gestor do PSE).
- Cadastrar ação: registrar código, nome, escola, tema do PSE, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial.
- Listar ações: exibir todas as ações cadastradas.
- Pesquisar ações: localizar registros por código, nome ou trecho da escola e nome ou trecho do tema.
- Atualizar situação: alterar a situação da ação para Planejada, Em andamento, Concluída ou Cancelada.
- Registrar participação: informar a quantidade efetiva de participantes após a conclusão da ação.
- Gerar resumo geral: apresentar totais de ações por situação, quantidade de participantes previstos e efetivos nas ações concluídas e percentual de participação.
- Validar informações: verificar campos obrigatórios, códigos duplicados, valores numéricos, formato de data, opções inválidas, registros inexistentes e capacidade máxima de armazenamento.

1.2 CÓDIGO DO PROJETO

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 100

// Vetores para armazenar os dados das acoes
int codigo[MAX];
char nome[MAX][100];
char escola[MAX][100];
char tema[MAX][100];
char data[MAX][20];
char responsavel[MAX][100];
char publico[MAX][100];
int previstos[MAX];
int efetivos[MAX];
int situacao[MAX];

int total = 0;

// Funcao para ler numeros inteiros
int lerInteiro() {
    int valor;

    while (scanf("%d", &valor) != 1) {
        printf("Valor invalido! Digite um numero: ");
        while (getchar() != '\n');
    }

    while (getchar() != '\n');

    return valor;
}

// Funcao para ler textos
void lerTexto(char texto[]) {
    scanf(" %99[^\n]", texto);
    while (getchar() != '\n');
}

// Funcao para validar a data DD/MM/AAAA
int validarData(char d[]) {
    int i;

    if (strlen(d) != 10)
        return 0;

    if (d[2] != '/' || d[5] != '/')
```

```
        return 0;

    for (i = 0; i < 10; i++) {
        if (i != 2 && i != 5) {
            if (d[i] < '0' || d[i] > '9')
                return 0;
        }
    }

    return 1;
}

// Funcao de login
int login() {
    char email[100];
    char senha[100];
    int tipo;

    printf("\n========== LOGIN PSE ==========\n");

    printf("E-mail: ");
    lerTexto(email);

    printf("Senha: ");
    lerTexto(senha);

    printf("\nTipo de usuario:\n");
    printf("1 - Diretor da escola\n");
    printf("2 - Profissional da saude\n");
    printf("3 - Professor/Equipe pedagogica\n");
    printf("4 - Coordenador/Gestor do PSE\n");

    printf("Escolha: ");
    tipo = lerInteiro();

    if (strcmp(email, "admin@pse.com") == 0 &&
        strcmp(senha, "1234") == 0 &&
        tipo >= 1 && tipo <= 4) {

        printf("\nLogin realizado com sucesso!\n");
        return 1;
    }

    printf("\nDados de login invalidos!\n");
```

```
    return 0;
}

// Funcao para localizar uma acao pelo codigo
int localizarAcao(int cod) {
    int i;

    for (i = 0; i < total; i++) {
        if (codigo[i] == cod) {
            return i;
        }
    }

    return -1;
}

// Funcao para cadastrar uma acao
void cadastrarAcao() {
    int cod;

    if (total >= MAX) {
        printf("\nLimite de cadastros atingido!\n");
        return;
    }

    printf("\n========== CADASTRAR ACAO ==========\n");

    printf("Codigo: ");
    cod = lerInteiro();

    if (localizarAcao(cod) != -1) {
        printf("Esse codigo ja esta cadastrado!\n");
        return;
    }

    if (cod <= 0) {
        printf("O codigo deve ser maior que zero!\n");
        return;
    }

    codigo[total] = cod;

    printf("Nome da acao: ");
    lerTexto(nome[total]);
```

```
    printf("Escola: ");
    lerTexto(escola[total]);

    printf("Tema do PSE: ");
    lerTexto(tema[total]);

    do {
        printf("Data (DD/MM/AAAA): ");
        scanf("%19s", data[total]);
        while (getchar() != '\n');

        if (!validarData(data[total])) {
            printf("Data invalida! Tente novamente.\n");
        }

    } while (!validarData(data[total]));

    printf("Publico-alvo: ");
    lerTexto(publico[total]);

    printf("Responsavel: ");
    lerTexto(responsavel[total]);

    do {
        printf("Quantidade prevista de participantes: ");
        previstos[total] = lerInteiro();

        if (previstos[total] < 0) {
            printf("A quantidade nao pode ser negativa!\n");
        }

    } while (previstos[total] < 0);

    // Situacao inicial: Planejada
    situacao[total] = 1;

    // Participacao ainda nao registrada
    efetivos[total] = -1;

    total++;

    printf("\nAcao cadastrada com sucesso!\n");
}
```

```
// Funcao para listar as acoes
void listarAcoes() {
    int i;

    printf("\n========== LISTA DE ACOES ==========\n");

    if (total == 0) {
        printf("Nao existem acoes cadastradas.\n");
        return;
    }

    for (i = 0; i < total; i++) {

        printf("\nCodigo: %d\n", codigo[i]);
        printf("Nome: %s\n", nome[i]);
        printf("Escola: %s\n", escola[i]);
        printf("Tema: %s\n", tema[i]);
        printf("Data: %s\n", data[i]);
        printf("Publico-alvo: %s\n", publico[i]);
        printf("Responsavel: %s\n", responsavel[i]);
        printf("Participantes previstos: %d\n", previstos[i]);

        printf("Situacao: ");

        if (situacao[i] == 1)
            printf("Planejada\n");
        else if (situacao[i] == 2)
            printf("Em andamento\n");
        else if (situacao[i] == 3)
            printf("Concluida\n");
        else
            printf("Cancelada\n");

        if (efetivos[i] != -1)
            printf("Participantes efetivos: %d\n", efetivos[i]);
        else
            printf("Participantes efetivos: Nao registrados\n");
    }
}

// Funcao para pesquisar acoes
void pesquisarAcao() {
    int opcao, cod, i, encontrou = 0;
```

```
char busca[100];

printf("\n========== PESQUISAR ACAO ==========\n");
printf("1 - Pesquisar por codigo\n");
printf("2 - Pesquisar por nome\n");
printf("3 - Pesquisar por escola\n");
printf("4 - Pesquisar por tema\n");
printf("Escolha: ");

opcao = lerInteiro();

if (opcao < 1 || opcao > 4) {
    printf("Opcao invalida!\n");
    return;
}

if (opcao == 1) {
    printf("Digite o codigo: ");
    cod = lerInteiro();

    i = localizarAcao(cod);

    if (i != -1) {
        printf("\nAcao encontrada: %s\n", nome[i]);
        printf("Escola: %s\n", escola[i]);
        printf("Tema: %s\n", tema[i]);
        printf("Data: %s\n", data[i]);
        printf("Codigo: %d\n", codigo[i]);
    } else {
        printf("Acao nao encontrada!\n");
    }

    return;
}

printf("Digite o termo da pesquisa: ");
lerTexto(busca);

for (i = 0; i < total; i++) {

    if ((opcao == 2 && strstr(nome[i], busca) != NULL) ||
        (opcao == 3 && strstr(escola[i], busca) != NULL) ||
        (opcao == 4 && strstr(tema[i], busca) != NULL)) {
```

```
            printf("\nCodigo: %d\n", codigo[i]);
            printf("Nome: %s\n", nome[i]);
            printf("Escola: %s\n", escola[i]);
            printf("Tema: %s\n", tema[i]);

            encontrou = 1;
        }
    }

    if (encontrou == 0) {
        printf("\nNenhuma acao encontrada!\n");
    }
}

// Funcao para atualizar a situacao
void atualizarSituacao() {
    int cod, pos, nova;

    printf("\n========== ATUALIZAR SITUACAO ==========\n");

    printf("Codigo da acao: ");
    cod = lerInteiro();

    pos = localizarAcao(cod);

    if (pos == -1) {
        printf("Acao nao encontrada!\n");
        return;
    }

    printf("\n1 - Planejada\n");
    printf("2 - Em andamento\n");
    printf("3 - Concluida\n");
    printf("4 - Cancelada\n");

    printf("Nova situacao: ");
    nova = lerInteiro();

    if (nova < 1 || nova > 4) {
        printf("Situacao invalida!\n");
        return;
    }

    situacao[pos] = nova;
```

```
    printf("\nSituacao atualizada com sucesso!\n");
}

// Funcao para registrar a participacao
void registrarParticipacao() {
    int cod, pos, quantidade;

    printf("\n========== REGISTRAR PARTICIPACAO ==========\n");

    printf("Codigo da acao: ");
    cod = lerInteiro();

    pos = localizarAcao(cod);

    if (pos == -1) {
        printf("Acao nao encontrada!\n");
        return;
    }

    if (situacao[pos] != 3) {
        printf("A acao precisa estar concluida!\n");
        return;
    }

    do {
        printf("Quantidade efetiva de participantes: ");
        quantidade = lerInteiro();

        if (quantidade < 0) {
            printf("Quantidade invalida!\n");
        }

    } while (quantidade < 0);

    efetivos[pos] = quantidade;

    printf("\nParticipacao registrada com sucesso!\n");
}

// Funcao para gerar o resumo geral
void gerarResumo() {
    int i;
    int planejadas = 0;
```

```
int andamento = 0;
int concluidas = 0;
int canceladas = 0;

int totalPrevistos = 0;
int totalEfetivos = 0;

float percentual = 0;

printf("\n========== RESUMO GERAL ==========\n");

if (total == 0) {
    printf("Nao existem dados para apresentar.\n");
    return;
}

for (i = 0; i < total; i++) {

    if (situacao[i] == 1)
        planejadas++;

    else if (situacao[i] == 2)
        andamento++;

    else if (situacao[i] == 3) {
        concluidas++;
        totalPrevistos += previstos[i];

        if (efetivos[i] != -1) {
            totalEfetivos += efetivos[i];
        }
    }

    else if (situacao[i] == 4)
        canceladas++;
}

if (totalPrevistos > 0) {
    percentual = (float)totalEfetivos /
                 totalPrevistos * 100;
}

printf("\nTotal de acoes: %d\n", total);
printf("Planejadas: %d\n", planejadas);
```

```
    printf("Em andamento: %d\n", andamento);
    printf("Concluidas: %d\n", concluidas);
    printf("Canceladas: %d\n", canceladas);

    printf("\nParticipantes previstos nas acoes concluidas: %d\n",
           totalPrevistos);

    printf("Participantes efetivos registrados: %d\n",
           totalEfetivos);

    printf("Percentual de participacao: %.2f%%\n",
           percentual);
}

// Programa principal
int main() {
    int opcao;
    int acesso = 0;

    printf("====================================\n");
    printf(" SISTEMA PSE - SAUDE NA ESCOLA\n");
    printf("====================================\n");

    // Permite tentar o login novamente
    while (acesso == 0) {
        acesso = login();
    }

    do {

        printf("\n========== MENU PRINCIPAL ==========\n");
        printf("1 - Cadastrar acao\n");
        printf("2 - Listar acoes\n");
        printf("3 - Pesquisar acao\n");
        printf("4 - Atualizar situacao\n");
        printf("5 - Registrar participacao\n");
        printf("6 - Gerar resumo geral\n");
        printf("0 - Sair\n");

        printf("\nEscolha uma opcao: ");
        opcao = lerInteiro();

        switch (opcao) {
```

```
            case 1:
                cadastrarAcao();
                break;

            case 2:
                listarAcoes();
                break;

            case 3:
                pesquisarAcao();
                break;

            case 4:
                atualizarSituacao();
                break;

            case 5:
                registrarParticipacao();
                break;

            case 6:
                gerarResumo();
                break;

            case 0:
                printf("\nEncerrando o sistema PSE...\n");
                break;

            default:
                printf("\nOpcao invalida! Tente novamente.\n");
        }

    } while (opcao != 0);

    return 0;
}
```

2. ENGENHARIA DE SOFTWARE

2.1 REQUISITOS FUNCIONAIS ATUALIZADOS

Em relação à primeira entrega, os requisitos funcionais foram revisados e ampliados para detalhar melhor as funcionalidades do sistema e adequá-las ao desenvolvimento da aplicação. Entre as principais mudanças estão a inclusão do login com e-mail, senha e tipo de usuário, a utilização de códigos para identificar as ações cadastradas, o detalhamento dos critérios de pesquisa e validação de informações e a ampliação dos indicadores apresentados no resumo geral. Essas alterações visam melhorar a organização dos registros, a identificação dos usuários e o acompanhamento das ações do PSE, mantendo o objetivo inicial do projeto.

Quadro 01 – Requisitos funcionais atualizados

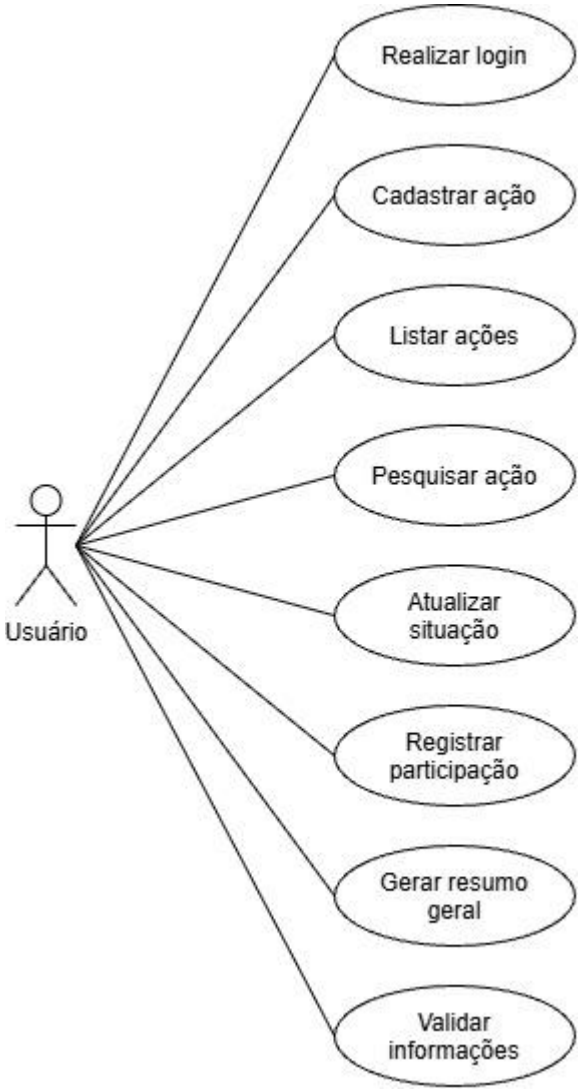
Código	Requisito	Descrição
RF01	Realizar login	O sistema deverá permitir que o usuário informe seu e-mail, senha e tipo de usuário, verificando se as informações correspondem a um cadastro existente para liberar o acesso ao sistema.
RF02	Cadastrar ação	O sistema deverá permitir o cadastro de uma nova ação do PSE, contendo informações como nome, tema, escola, data, responsável e quantidade prevista de participantes.
RF03	Listar ações	O sistema deve exibir todas as ações cadastradas durante a execução.
RF04	Pesquisar ação	O sistema deve permitir pesquisa por código, escola ou tema.
RF05	Atualizar situação	O sistema deverá permitir alterar a situação de uma ação entre Planejada, Em andamento, Concluída ou Cancelada.
RF06	Registrar participação	O sistema deverá permitir registrar a quantidade de participantes de uma ação após sua realização.

RF07	Gerar resumo geral	O sistema deve apresentar quantidades por situação, totais de participantes e percentual de participação nas ações realizadas.
RF08	Validar informações	O sistema deve validar campos obrigatórios, código duplicado, valores numéricos, opções do menu, registros inexistentes e limite de armazenamento.

Fonte: elaborado pelos autores, 2026

2.2 DIAGRAMA DE CASOS DE USO

Figura 01 – Diagrama de Casos de Uso



Fonte: elaborado pelos autores no draw.io, 2026

2.3 DESCRIÇÃO DOS CASOS DE USO PRINCIPAIS

UC01 – Realizar login

OBJETIVO: Autenticar e identificar o usuário que deseja acessar o sistema.

ATOR: Usuário (diretor da escola, profissional da saúde, professor/equipe pedagógica ou coordenador/gestor do PSE).

PRÉ-CONDIÇÕES: O usuário deve possuir cadastro no sistema, contendo e-mail, senha e tipo de usuário.

FLUXO PRINCIPAL:

1. O usuário inicia o sistema.
2. O sistema solicita e-mail, senha e tipo de usuário.
3. O usuário informa os dados solicitados.
4. O sistema verifica se as informações correspondem a um cadastro existente.
5. O sistema libera o acesso ao menu principal.

FLUXOS ALTERNATIVOS: Caso o e-mail, a senha ou o tipo de usuário estejam incorretos, o sistema informa que os dados são inválidos e solicita uma nova tentativa.

PÓS-CONDIÇÕES: O usuário é autenticado e pode acessar as funcionalidades do sistema.

UC02 – Cadastrar ação

OBJETIVO: Registrar uma nova ação do PSE.

ATOR: Usuário.

PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema.

FLUXO PRINCIPAL:

1. O usuário seleciona a opção de cadastro.
2. O sistema solicita os dados da ação.
3. O usuário informa o código, nome, tema, escola, data, responsável e quantidade prevista de participantes.
4. O sistema valida as informações.
5. O sistema cadastra a ação.

FLUXOS ALTERNATIVOS: Caso existam campos obrigatórios vazios, código duplicado ou valores inválidos, o sistema informa o problema e solicita a correção.

PÓS-CONDIÇÕES: A ação é cadastrada e fica disponível para consulta.

UC03 – Listar ações

OBJETIVO: Visualizar todas as ações cadastradas.

ATOR: Usuário.

PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema.

FLUXO PRINCIPAL:

1. O usuário seleciona a opção de listar ações.
2. O sistema apresenta as ações cadastradas e suas informações.

FLUXOS ALTERNATIVOS: Caso não existam ações cadastradas, o sistema informa que não há registros disponíveis.

PÓS-CONDIÇÕES: As ações cadastradas são exibidas ao usuário.

UC04 – Pesquisar ação

OBJETIVO: Localizar uma ação cadastrada.

ATOR: Usuário.

PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema.

FLUXO PRINCIPAL:

1. O usuário seleciona a opção de pesquisa.
2. O sistema solicita o critério de pesquisa.
3. O usuário informa o código, a escola ou o tema.
4. O sistema realiza a busca e apresenta os resultados.

FLUXOS ALTERNATIVOS: Caso nenhuma ação seja encontrada, o sistema informa que não existem registros correspondentes.

PÓS-CONDIÇÕES: A ação localizada é apresentada ao usuário.

UC05 – Atualizar situação

OBJETIVO: Alterar a situação de uma ação cadastrada.

ATOR: Usuário.

PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema e a ação deve estar cadastrada.

FLUXO PRINCIPAL:

1. O usuário seleciona a opção de atualizar situação.
2. O sistema solicita o código da ação.
3. O usuário informa o código.

4. O sistema apresenta a ação encontrada.
5. O usuário seleciona a nova situação.
6. O sistema atualiza o registro.

FLUXOS ALTERNATIVOS: Caso o código não seja encontrado ou a situação informada seja inválida, o sistema apresenta uma mensagem de erro.
PÓS-CONDIÇÕES: A situação da ação é atualizada.

UC06 – Registrar participação

OBJETIVO: Registrar a quantidade de participantes de uma ação após sua realização.
ATOR: Usuário.
PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema e a ação deve estar cadastrada e concluída.
FLUXO PRINCIPAL:

1. O usuário seleciona a opção de registrar participação.
2. O sistema solicita o código da ação.
3. O usuário informa o código.
4. O sistema localiza a ação.
5. O usuário informa a quantidade efetiva de participantes.
6. O sistema valida e registra a quantidade.

FLUXOS ALTERNATIVOS: Caso a ação não seja encontrada ou a quantidade informada seja inválida, o sistema apresenta uma mensagem de erro.
Pós-condições: A quantidade de participantes fica registrada na ação.

UC07 – Gerar resumo geral

OBJETIVO: Apresentar informações gerais sobre as ações cadastradas.
ATOR: Usuário.
PRÉ-CONDIÇÕES: O usuário deve estar identificado no sistema.
FLUXO PRINCIPAL:

1. O usuário seleciona a opção de gerar resumo.
2. O sistema reúne as informações das ações cadastradas.
3. O sistema apresenta as quantidades de ações por situação, o total de participantes e o percentual de participação nas ações realizadas.

FLUXOS ALTERNATIVOS: Caso não existam ações cadastradas, o sistema informa que não há dados disponíveis para o resumo.
PÓS-CONDIÇÕES: O resumo geral é apresentado ao usuário.

3. PRODUÇÃO TEXTUAL – METEP

3.1 MANUAL DO USUÁRIO

3.1.1 Apresentação do sistema

O sistema PSE – Saúde Bucal foi desenvolvido com o objetivo de auxiliar no cadastro, na organização e no acompanhamento das ações relacionadas ao Programa Saúde na Escola (PSE), com foco na temática de saúde bucal.

A aplicação permite registrar ações, consultar informações, acompanhar a situação de cada atividade e registrar a quantidade de participantes. Dessa forma, busca facilitar o controle das atividades realizadas pelas escolas e pelos profissionais envolvidos no programa.

O sistema foi desenvolvido na linguagem C, utilizando o ambiente Dev-C++, com estruturas condicionais, vetores, laços de repetição e funções para organizar as operações.

3.1.2 Instruções para compilação e execução

Para utilizar o sistema, é necessário possuir o programa Dev-C++ instalado no computador. O procedimento para executar o sistema é:

1. Abrir o Dev-C++.
2. Abrir o arquivo com o código-fonte do sistema ou criar um novo projeto em linguagem C.
3. Inserir o código-fonte no editor.
4. Salvar o arquivo.

5. Clicar na opção Executar e Compilar ou pressionar a tecla F11.
6. Após a compilação, o programa será executado e apresentará a tela de acesso.

Na tela inicial, o usuário deverá informar os dados solicitados para realizar o login e acessar o menu principal.

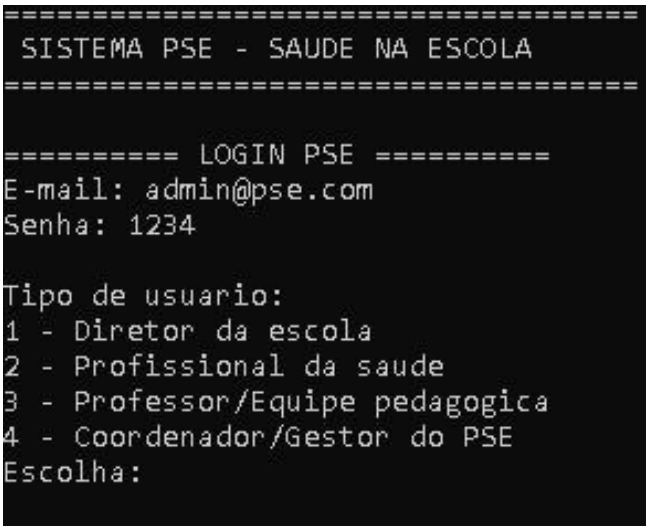
3.1.3 Funcionalidades do sistema

O sistema apresenta as seguintes funcionalidades:
1. Realizar login: permite que o usuário informe e-mail, senha e tipo de usuário para acessar o sistema.
2. Cadastrar ação: possibilita registrar uma nova ação, informando código, nome, escola, tema do PSE, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial.
3. Listar ações: exibe as ações cadastradas durante a execução do programa, permitindo visualizar as informações registradas.
4. Pesquisar ações: permite localizar uma ação por meio de seu código, nome, escola ou tema, facilitando a consulta dos registros.
5. Atualizar situação: permite alterar a situação de uma ação cadastrada para uma das seguintes opções: Planejada, Em andamento, Concluída ou Cancelada.
6. Registrar participação: possibilita informar a quantidade efetiva de participantes de uma ação após sua conclusão.
7. Gerar resumo geral: apresenta informações gerais sobre as ações cadastradas, incluindo a quantidade de ações por situação, o número de participantes previstos e efetivos e o percentual de participação.

3.1.4 Exemplos de utilização e capturas de tela

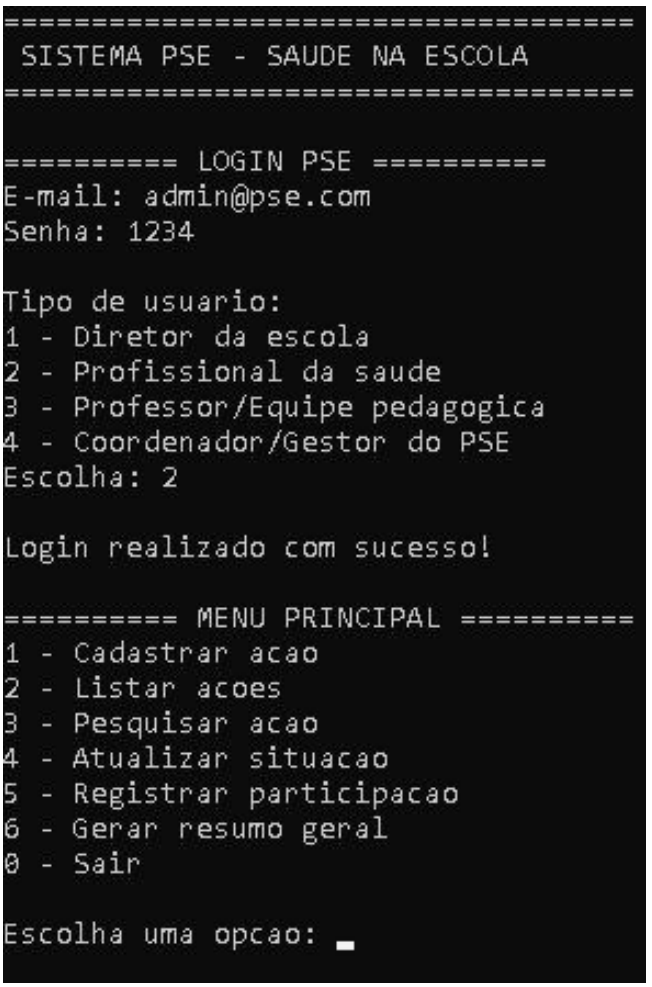
A seguir, são apresentadas as capturas de tela que demonstram o funcionamento das principais operações do sistema.

Figura 02 – Tela de acesso ao sistema



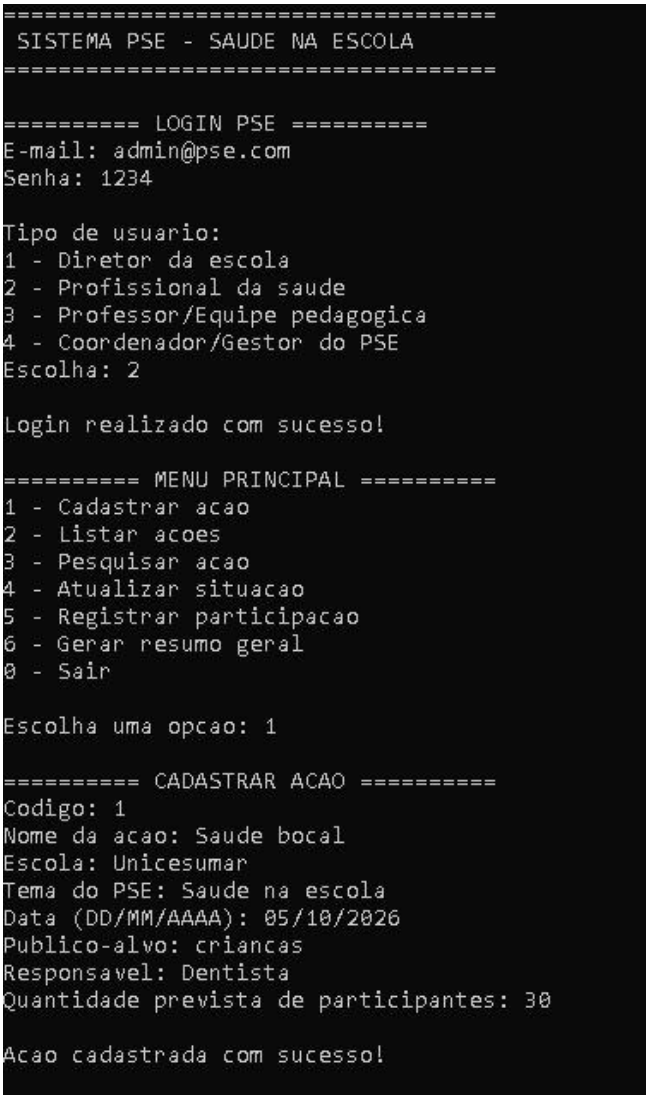
Fonte: elaborado pelos autores, 2026

Figura 03 – Menu principal



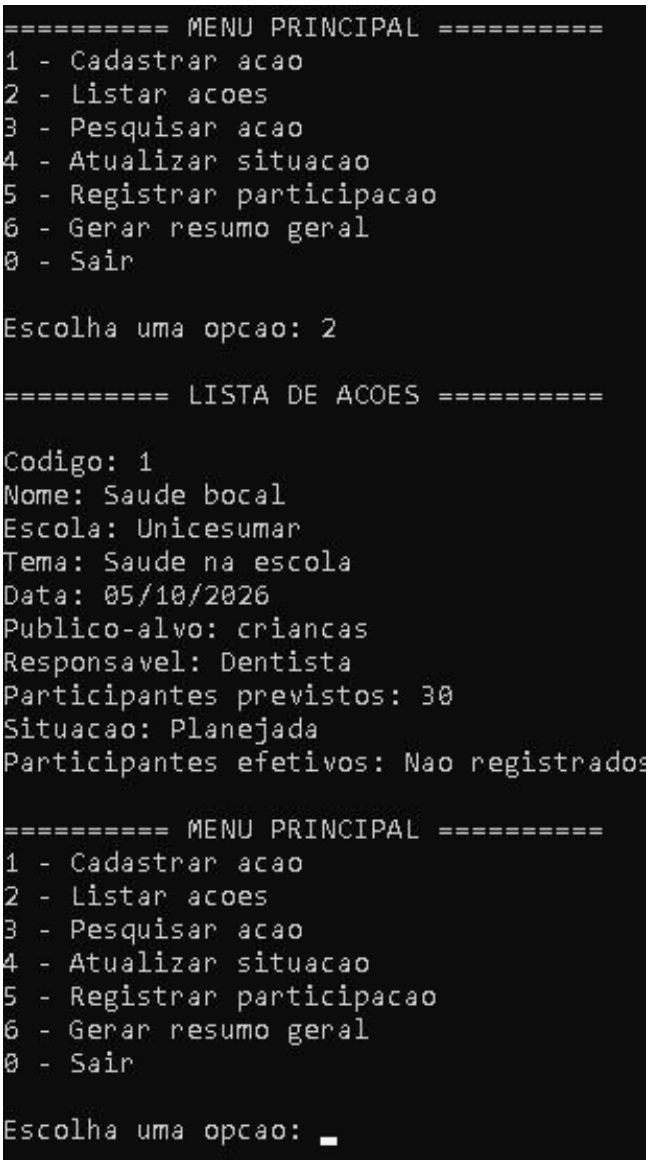
Fonte: elaborado pelos autores, 2026

Figura 04 – Cadastro de uma ação



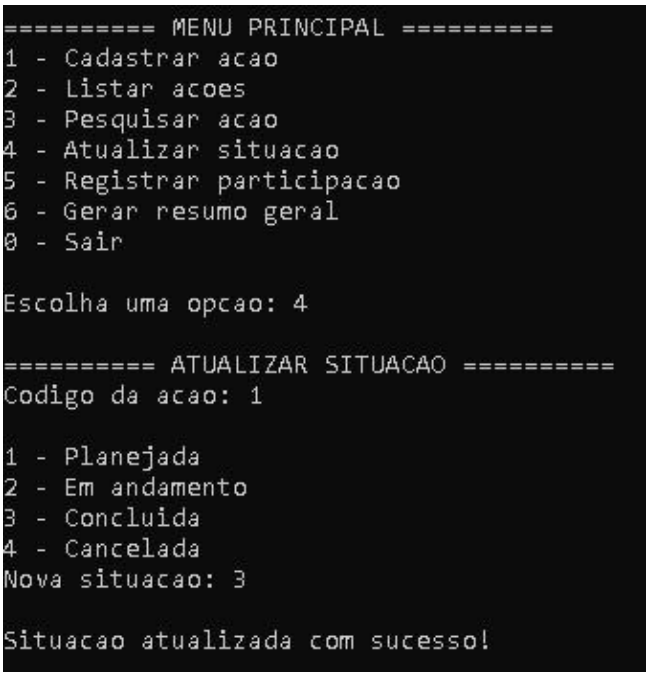
Fonte: elaborado pelos autores, 2026

Figura 05 – Listagem e pesquisa de ações



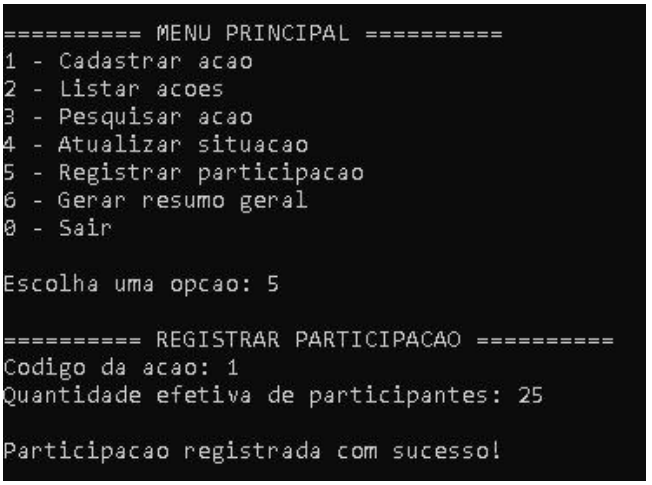
Fonte: elaborado pelos autores, 2026

Figura 06 – Atualização da situação da ação



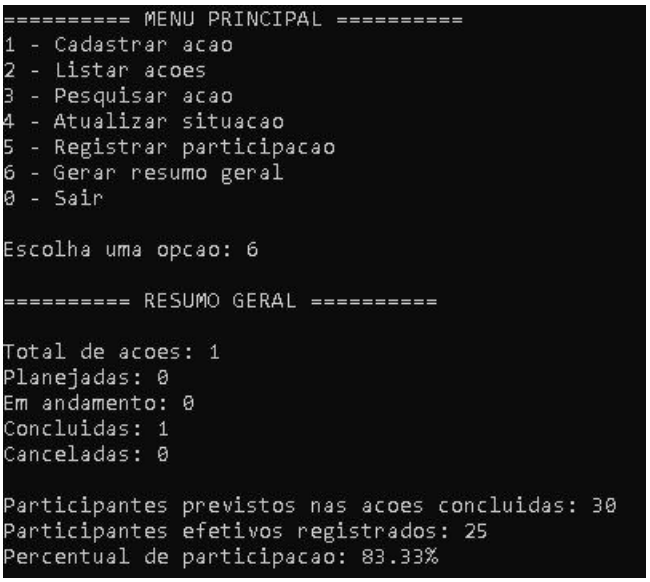
Fonte: elaborado pelos autores, 2026

Figura 07 – Registro de participação



Fonte: elaborado pelos autores, 2026

Figura 08 – Resumo geral



Fonte: elaborado pelos autores, 2026

3.2 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema PSE – Saúde Bucal possibilitou aplicar, na prática, os conhecimentos adquiridos durante as aulas de Linguagem e Técnicas de Programação e Engenharia de Software.

Durante o desenvolvimento, foram trabalhados conceitos da linguagem C, como variáveis, estruturas condicionais, vetores, laços de repetição e funções. Esses recursos permitiram construir um programa organizado, com diferentes funcionalidades para o gerenciamento das ações.

Os testes realizados demonstraram o funcionamento das principais operações, como cadastro, listagem, pesquisa, atualização de situação, registro de participação e geração de resumo geral.

Como limitação, os dados cadastrados ficam armazenados apenas durante a execução do programa. Portanto, ao encerrá-lo, as informações são perdidas. Como possibilidade de melhoria, futuramente o sistema poderá contar com armazenamento permanente de dados, cadastro de diferentes usuários e uma interface gráfica mais intuitiva.

Conclui-se que o projeto contribuiu para o aprendizado prático da programação e para a compreensão de como a tecnologia pode auxiliar na organização e no acompanhamento das ações do Programa Saúde na Escola.

4. GITHUB

Link do repositório no GitHub da AEP2: https://github.com/karencassero/AEP2-PSE

5. VÍDEO

Link do vídeo: https://www.youtube.com/watch?v=RgpL2U4h8OM&t=1s